

UNIDAD 00

ACTIVIDADES 1-15



Sistema de Gestión Empresarial

PYTHON 3 BÀSIC

Actividad 1	3
Actividad 2	6
Actividad 3	8
Actividad 4	10
Actividad 5	13
Actividad 6	20
Actividad 7	24
Actividad 8	27
Actividad 9	30
Actividad 10	35
Actividad 11	39
Actividad 12	44
Actividad 13	46
Actividad 14	48
Actividad 15	51

Actividad 1

```
from functools import reduce

# Lista de diccionarios componentes
Componentes = [
    {"nombre": "teclado", "precio": 25},
    {"nombre": "Raton", "precio": 5},
    {"nombre": "Ordenador", "precio": 2500},
    {"nombre": "Pantalla", "precio": 100}
]

# Buscar producto más barato con lambda
componenteBarato = min(Componentes, key=lambda x: x["precio"])
print(componenteBarato)

# Extraer los nombres de los componentes
ComponentesNombres = list(map(lambda x: x["nombre"], Componentes))
print(ComponentesNombres)

# Convertir la cadena en una lista de caracteres
UnicoAccesorio = "Alfombrilla"
print(list(UnicoAccesorio))

# Convertir a entero
numeroTexto = 3
numeroEntero = int(numeroTexto)
print(numeroEntero)

# Cadena de números a lista de enteros
CadenaNumeros = "1,12,3,14,15,6,17,8,19"
listaEnteros = list(map(int, CadenaNumeros.split(",")))
print(listaEnteros)

# Filtrar números mayores a 10 (convertimos a lista para imprimir los valores reales)
Uso_filter = list(filter(lambda x: x > 10, listaEnteros))
print(Uso_filter) # Imprime numeroa mayores a 10

# Multiplicar todos los números de la lista
Uso_reduce = reduce(lambda x, y: x * y, listaEnteros)
```

```
print(Uso_reduce) # Imprime la multiplicación de todos los números de la lista
```

uso sencillo de lambda:

Buscar el componente más barato de la lista, usando la key lambda.

con un min(), apuntamos al mínimo y con lambda se coge el valor de precio.

```
# Lista de diccionarios componentes
Componentes = [
    {"nombre": "teclado", "precio": 25},
    {"nombre": "Raton", "precio": 5},
    {"nombre": "Ordenador", "precio": 2500},
    {"nombre": "Pantalla", "precio": 100}
]

# Buscar producto más barato con lambda
componenteBarato = min(Componentes, key=lambda x: x["precio"])
print(componenteBarato)
```

Resultado:

```
{'nombre': 'Raton', 'precio': 5}
```

Ejemplos del map() list() e int() :

list() : Convierte en lista

map() ; Coge un conjunto de elementos para añadirle restricciones de manera conjunta

int : Transforma a entero

```
# Extraer los nombres de los componentes
ComponentesNombres = list(map(lambda x: x["nombre"], Componentes))
print(ComponentesNombres)

# Convertir la cadena en una lista de caracteres
UnicoAccesorio = "Alfombrilla"
print(list(UnicoAccesorio))

# Convertir a entero
numeroTexto = 3
numeroEntero = int(numeroTexto)
print(numeroEntero)
```

Resultado:

```
['teclado', 'Raton', 'Ordenador', 'Pantalla']
['A', 'l', 'f', 'o', 'm', 'b', 'r', 'i', 'l', 'l', 'a']
3
```

Crear una cadena de texto con espacios y convertirla en lista de enteros y con filter() eliminar los números menores de 10.

```
# Cadena de números a lista de enteros
CadenaNumeros = "1,12,3,14,15,6,17,8,19"
listaEnteros = list(map(int, CadenaNumeros.split(",")))
print(listaEnteros)
```

Resultado:

```
[1, 12, 3, 14, 15, 6, 17, 8, 19]
[12, 14, 15, 17, 19]
```

Con la cadena restante, con el reduce(), multiplicar todos los numeros

importar el functools para poder usar el reduce

```
from functools import reduce
```

```
# Multiplicar todos los números de la lista
Uso_reduce = reduce(lambda x, y: x * y, listaEnteros)
print(Uso_reduce) # Imprime la multiplicación de todos los números de la lista
```

Resultado:

```
117210240
```

Actividad 2

```
sudoku_valido = (  
    "5 3 4 6 7 8 9 1 2\n"  
    "6 7 2 1 9 5 3 4 8\n"  
    "1 9 8 3 4 2 5 6 7\n"  
    "8 5 9 7 6 1 4 2 3\n"  
    "4 2 6 8 5 3 7 9 1\n"  
    "7 1 3 9 2 4 8 5 6\n"  
    "9 6 1 5 3 7 2 8 4\n"  
    "2 8 7 4 1 9 6 3 5\n"  
    "3 4 5 2 8 6 1 7 9\n"  
)  
  
with open("Sudoku.in", "w") as archivo_creado:  
    archivo_creado.write(sudoku_valido)  
  
def esSudokuCorrecto(miArrayBi):  
    # Comprueba que cada fila contenga los numeros  
    for fila in miArrayBi:  
        if sorted(fila) != list(range(1, 10)):  
            return False  
  
    # Comprueba que cada columna contenga los numeros  
    for c in range(9):  
        columna = [miArrayBi[f][c] for f in range(9)]  
        if sorted(columna) != list(range(1, 10)):  
            return False  
  
    # Comprueba que cada bloque de 3x3 contenga los numeros  
    for f in range(0, 9, 3):  
        for c in range(0, 9, 3):  
            bloque = [miArrayBi[f+i][c+j] for i in range(3) for j in  
range(3)]  
            if sorted(bloque) != list(range(1, 10)):  
                return False  
  
    return True  
  
# Lee el tablero directamente del archivo  
matriz = []  
with open("Sudoku.in", "r") as archivo:  
    for linea in archivo:  
        if linea.strip():  
            matriz.append([int(n) for n in linea.split()])  
  
# Muestra el contenido del Sudoku por pantalla
```

```
print("Tablero cargado:")
for fila in matriz:
    print(*fila)

print() # Linea en blanco para separar
```

Comprobación de filas y columnas :

```
def esSudokuCorrecto(miArrayBi):
    # Comprueba que cada fila contenga los numeros
    for fila in miArrayBi:
        if sorted(fila) != list(range(1, 10)):
            return False

    # Comprueba que cada columna contenga los numeros
    for c in range(9):
        columna = [miArrayBi[f][c] for f in range(9)]
        if sorted(columna) != list(range(1, 10)):
            return False

    # Comprueba que cada bloque de 3x3 contenga los numeros
    for f in range(0, 9, 3):
        for c in range(0, 9, 3):
            bloque = [miArrayBi[f+i][c+j] for i in range(3) for j in range(3)]
            if sorted(bloque) != list(range(1, 10)):
                return False

    return True
```

Leer tablero e imprimir:

```
# Lee el tablero directamente del archivo
matriz = []
with open("Sudoku.in", "r") as archivo:
    for linea in archivo:
        if linea.strip():
            matriz.append([int(n) for n in linea.split()])

# Muestra el contenido del Sudoku por pantalla
print("Sudoku:")
for fila in matriz:
    print(*fila)

print() # Linea en blanco para separar
```

Resultado:

```
Sudoku:
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9
```

Actividad 3

```
# Lista de patrones a buscar
patrones = ["00", "101", "ABC", "HO"]

def numeroPatrones(text):
    text = text.upper()
    numero_patrones = 0
    no_patron = 0

    i = 0
    while i < len(text):
        encontrado = False

        # Comprobamos si en la posición actual empieza algún patrón o su
        # opuesto
        for patron in patrones:
            patron_opuesto = patron[::-1]
            longitud = len(patron)

            # Subcadena actual del tamaño del patrón
            subcadena = text[i : i + longitud]

            if subcadena == patron or subcadena == patron_opuesto:
                numero_patrones += 1
                i += longitud # Saltamos el patrón completo encontrado
                encontrado = True
                break
```

```

        # Si en esta posición no encuentra ningún patron, suma 1
        if not encontrado:
            no_patron += 1
            i += 1

    print(f"Patrones encontrados: {numero_patrones}")
    print(f"Elementos que NO son patrón: {no_patron}")

    return numero_patrones

# Programa principal
cadenaTexto = input("Introduce una cadena de texto: ")
numeroPatrones(cadenaTexto)

```

**Solicita una cadena de texto que dice cuantas veces saca los patrones siguientes: :
 “00” “101”, “ABC”, “HO” y que diga el numero de patrones que han salido y los
 opuestos con un metodo.**

Se recorre con for (usando in)

```

def numeroPatrones(text):
    text = text.upper()
    numero_patrones = 0
    no_patron = 0

    i = 0
    while i < len(text):
        encontrado = False

        # Comprobamos si en la posición actual empieza algún patrón o su opuesto
        for patron in patrones:
            patron_opuesto = patron[::-1]
            longitud = len(patron)

            # Subcadena actual del tamaño del patrón
            subcadena = text[i : i + longitud]

            if subcadena == patron or subcadena == patron_opuesto:
                numero_patrones += 1
                i += longitud # Saltamos el patrón completo encontrado
                encontrado = True
                break

        # Si en esta posición no encuentra ningún patron, suma 1
        if not encontrado:
            no_patron += 1
            i += 1

```

Resultado:

```
Introduce una cadena de texto: ABC000A
Patrones encontrados: 2
Elementos que NO son patrón: 2
```

Actividad 4

```
def contandoMinas(campo):
    filas = len(campo)
    columnas = len(campo[0])

    resultado = []

    for i in range(filas):
        nuevaFila = []
        for x in range(columnas):

            # Si hay mina, se queda en -1
            if campo[i][x] == -1:
                nuevaFila.append(-1)
            else:
                contadorMinas = 0

                # Aquí se miran las posiciones de alrededor
                for fila_conjunta in range(i - 1, i + 2):
                    for columna_conjunta in range(x - 1, x + 2):

                        # añadir límite superior
                        if 0 <= fila_conjunta < filas and 0 <= columna_conjunta < columnas:
                            if campo[fila_conjunta][columna_conjunta] == -1:
                                contadorMinas += 1

                nuevaFila.append(contadorMinas)

        resultado.append(nuevaFila)
    return resultado
```

```
campo = [  
    [ 0, -1,  0,  0],  
    [ 0,  0, -1,  0],  
    [-1,  0,  0,  0]  
]  
  
tablero_resuelto = contandoMinas(campo)  
  
# Mostramos el resultado  
for fila in tablero_resuelto:  
    print(fila)
```

Ejemplo de buscaminas, recorrido por for mirando en sus diferentes posiciones (El campo del buscaminas lo modificamos nosotros mismo y ponemos la mina (-1) donde se quiera).

```
def contandoMinas(campo):  
    filas = len(campo)  
    columnas = len(campo[0])  
  
    resultado = []  
  
    for i in range(filas):  
        nuevaFila = []  
        for x in range(columnas):  
            # Si hay mina, se queda en -1  
            if campo[i][x] == -1:  
                nuevaFila.append(-1)  
            else:  
                contadorMinas = 0  
  
                # Aquí se miran las posiciones de alrededor  
                for fila_conjunta in range(i - 1, i + 2):  
                    for columna_conjunta in range(x - 1, x + 2):  
                        # Corregido: añadir límite superior (< filas y < columnas)  
                        if 0 <= fila_conjunta < filas and 0 <= columna_conjunta < columnas:  
                            if campo[fila_conjunta][columna_conjunta] == -1:  
                                contadorMinas += 1  
  
                nuevaFila.append(contadorMinas)  
  
        resultado.append(nuevaFila)  
    return resultado
```

Campo :

```
# Ejemplo de uso
campo = [
    [ 0, -1,  0,  0],
    [ 0,  0, -1,  0],
    [-1,  0,  0,  0]
]

tablero_resuelto = contandoMinas(campo)

# Mostramos el resultado
for fila in tablero_resuelto:
    print(fila)
```

Resultado:

```
[1, -1, 2, 1]
[2, 3, -1, 1]
[-1, 2, 1, 1]
```

Actividad 5

```
class Profesor:
    # Atributos del profesor
    def __init__(self, dni, nombre, tipo):
        self.dni = dni
        self.nombre = nombre
        self.tipo = tipo

class Alumno:
    # Atributos del alumno y asigna un profesor responsable opcional
    def __init__(self, id_alumno, nombre, curso,
profesor_responsable=None):
        self.id_alumno = id_alumno
        self.nombre = nombre
        self.curso = curso
        self.profesor_responsable = profesor_responsable

class Escuela:
    # Datos de la escuela y las listas de profesores y alumnos
    def __init__(self, nombre, localidad, responsable):
        self.nombre = nombre
        self.localidad = localidad
        self.responsable = responsable
        self.profesores = []
        self.alumnos = []

    # Profesores

    # Crea y añade un nuevo profesor a la lista
    def anadir_profesor(self, dni, nombre, tipo):
        prof_nuevo = Profesor(dni, nombre, tipo)
        self.profesores.append(prof_nuevo)
        print("Profesor añadido correctamente.")

    # Busca y devuelve un profesor por su dni
    def buscar_profesor(self, dni):
        for prof_actual in self.profesores:
            if prof_actual.dni == dni:
                return prof_actual
        return None
```

```
# Modifica los datos de un profesor existente
def modificar_profesor(self, dni, nuevo_nombre, nuevo_tipo):
    prof_encontrado = self.buscar_profesor(dni)
    if prof_encontrado:
        if nuevo_nombre:
            prof_encontrado.nombre = nuevo_nombre
        if nuevo_tipo:
            prof_encontrado.tipo = nuevo_tipo
        print("Profesor modificado.")
    else:
        print("Profesor no encontrado.")

# Elimina un profesor y desasigna sus alumnos tutorizados
def eliminar_profesor(self, dni):
    prof_encontrado = self.buscar_profesor(dni)
    if prof_encontrado:
        for alumno_actual in self.alumnos:
            if alumno_actual.profesor_responsable == prof_encontrado:
                alumno_actual.profesor_responsable = None
        self.profesores.remove(prof_encontrado)
        print("Profesor eliminado.")
    else:
        print("Profesor no encontrado.")

# Alumnos

# Crea y añade un nuevo alumno asignándole un profesor tutor si se indica
def anadir_alumno(self, id_alumno, nombre, curso, dni_profesor=None):
    prof_tutor = self.buscar_profesor(dni_profesor) if dni_profesor else None
    alumno_nuevo = Alumno(id_alumno, nombre, curso, prof_tutor)
    self.alumnos.append(alumno_nuevo)
    print("Alumno añadido correctamente.")

# Busca y devuelve un alumno por su id
def buscar_alumno(self, id_alumno):
    for alumno_actual in self.alumnos:
        if alumno_actual.id_alumno == id_alumno:
            return alumno_actual
```

```
        return None

    # Modifica los datos de un alumno existente
    def modificar_alumno(self, id_alumno, nuevo_nombre, nuevo_curso,
nuevo_dni_prof):
        alumno_encontrado = self.buscar_alumno(id_alumno)
        if alumno_encontrado:
            if nuevo_nombre:
                alumno_encontrado.nombre = nuevo_nombre
            if nuevo_curso:
                alumno_encontrado.curso = nuevo_curso
            if nuevo_dni_prof:
                alumno_encontrado.profesor_responsable =
self.buscar_profesor(nuevo_dni_prof)
            print("Alumno modificado.")
        else:
            print("Alumno no encontrado.")

    # Elimina un alumno de la lista
    def eliminar_alumno(self, id_alumno):
        alumno_encontrado = self.buscar_alumno(id_alumno)
        if alumno_encontrado:
            self.alumnos.remove(alumno_encontrado)
            print("Alumno eliminado.")
        else:
            print("Alumno no encontrado.")

    # Muestra por pantalla toda la información de la escuela, profesores
y alumnos
    def mostrar_todo(self):
        print(f"\n Escuela: {self.nombre} ({self.localidad}) | Resp:
{self.responsable} ---")

        print("Profesores:")
        if not self.profesores:
            print(" (Ninguno)")
        for prof_item in self.profesores:
            print(f" - DNI: {prof_item.dni} | Nombre:
{prof_item.nombre} | Tipo: {prof_item.tipo}")

        print("Alumnos:")
```

```

        if not self.alumnos:
            print("  (Ninguno)")
        for alum_item in self.alumnos:
            nombre_tutor = alum_item.profesor_responsable.nombre if
alum_item.profesor_responsable else "Sin tutor"
            print(f"  - ID: {alum_item.id_alumno} | Nombre:
{alum_item.nombre} | Curso: {alum_item.curso} | Tutor: {nombre_tutor}")

```

Crear una clase escuela, que gestione diversas escuelas con profesores alumnos, etc...

Restricciones:

- No hay un alumno que vaya a dos escuelas distintas
- No hay dos profesores que trabajan en diferentes escuelas.
- Se deben representar las relaciones entre clases.
- Deben tener métodos para gestionar correctamente todos los elementos.

Ejemplos de clases :

```

class Profesor:
    # Atributos del profesor
    def __init__(self, dni, nombre, tipo):
        self.dni = dni
        self.nombre = nombre
        self.tipo = tipo

class Alumno:
    # Atributos del alumno y asigna un profesor responsable opcional
    def __init__(self, id_alumno, nombre, curso, profesor_responsable=None):
        self.id_alumno = id_alumno
        self.nombre = nombre
        self.curso = curso
        self.profesor_responsable = profesor_responsable

class Escuela:
    # Datos de la escuela y las listas de profesores y alumnos
    def __init__(self, nombre, localidad, responsable):
        self.nombre = nombre
        self.localidad = localidad
        self.responsable = responsable
        self.profesores = []
        self.alumnos = []

```

Main:

```

from UD00_PE5_Gestor_Escuela import Escuela

```

```
# Crear la escuela inicial
escuela = Escuela("IES Fasuti Barbera", "Alaquas", "Fausti Barbera")

opcion = ""

while opcion != "0":
    print("\n Gestion Escuela ")
    print("1. Añadir profesor")
    print("2. Modificar profesor")
    print("3. Eliminar profesor")
    print("4. Añadir alumno")
    print("5. Modificar alumno")
    print("6. Eliminar alumno")
    print("7. Mostrar todo")
    print("0. Salir")

    opcion = input("Elige una opción: ")

    if opcion == "1":
        dni = input("DNI: ")
        nombre = input("Nombre: ")
        tipo = input("Tipo (ciencias/letras/mixto): ")
        escuela.anadir_profesor(dni, nombre, tipo)

    elif opcion == "2":
        dni = input("DNI del profesor a modificar: ")
        nombre = input("Nuevo nombre (deja en blanco para no cambiar): ")
        tipo = input("Nuevo tipo (deja en blanco para no cambiar): ")
        escuela.modificar_profesor(dni, nombre, tipo)

    elif opcion == "3":
        dni = input("DNI del profesor a eliminar: ")
        escuela.eliminar_profesor(dni)

    elif opcion == "4":
        id_a = input("ID alumno: ")
        nombre = input("Nombre: ")
        curso = input("Curso: ")
        dni_p = input("DNI del profesor asignado (opcional): ")
        escuela.anadir_alumno(id_a, nombre, curso, dni_p)

    elif opcion == "5":
```

```
        id_a = input("ID del alumno a modificar: ")
        nombre = input("Nuevo nombre (deja en blanco para no cambiar): ")
    ")
        curso = input("Nuevo curso (deja en blanco para no cambiar): ")
        dni_p = input("Nuevo DNI de profesor (deja en blanco para no
cambiar): ")
        escuela.modificar_alumno(id_a, nombre, curso, dni_p)

    elif opcion == "6":
        id_a = input("ID del alumno a eliminar: ")
        escuela.eliminar_alumno(id_a)

    elif opcion == "7":
        escuela.mostrar_todo()

    elif opcion == "0":
        print("Saliendo del programa...")
    else:
        print("Opción no válida.")
```

Main con un “switch” if, elif, else

Importando la clase con from Clase import Clase

```
from UD00_PE5_Gestor_Escuela import Escuela

# Crear la escuela inicial
escuela = Escuela("IES Fasuti Barbera", "Alaquas", "Fausti Barbera")

opcion = ""

while opcion != "0":
    print("\n=== MENÚ GESTIÓN ESCUELA ===")
    print("1. Añadir profesor")
    print("2. Modificar profesor")
    print("3. Eliminar profesor")
    print("4. Añadir alumno")
    print("5. Modificar alumno")
    print("6. Eliminar alumno")
    print("7. Mostrar todo")
    print("0. Salir")

    opcion = input("Elige una opción: ")

    if opcion == "1":
        dni = input("DNI: ")
        nombre = input("Nombre: ")
        tipo = input("Tipo (ciencias/letras/mixto): ")
        escuela.anadir_profesor(dni, nombre, tipo)

    elif opcion == "2":
        dni = input("DNI del profesor a modificar: ")
        nombre = input("Nuevo nombre (deja en blanco para no cambiar): ")
        tipo = input("Nuevo tipo (deja en blanco para no cambiar): ")
        escuela.modificar_profesor(dni, nombre, tipo)

    elif opcion == "3":
        dni = input("DNI del profesor a eliminar: ")
        escuela.eliminar_profesor(dni)

    elif opcion == "4":
        id_a = input("ID alumno: ")
        nombre = input("Nombre: ")
        curso = input("Curso: ")
        dni_p = input("DNI del profesor asignado (opcional): ")
        escuela.anadir_alumno(id_a, nombre, curso, dni_p)

    elif opcion == "5":
        id_a = input("ID del alumno a modificar: ")
        nombre = input("Nuevo nombre (deja en blanco para no cambiar): ")
        curso = input("Nuevo curso (deja en blanco para no cambiar): ")
        dni_p = input("Nuevo DNI de profesor (deja en blanco para no cambiar): ")
        escuela.modificar_alumno(id_a, nombre, curso, dni_p)

    elif opcion == "6":
        id_a = input("ID del alumno a eliminar: ")
        escuela.eliminar_alumno(id_a)

    elif opcion == "7":
        escuela.mostrar_todo()

    elif opcion == "0":
        print("Saliendo del programa...")
    else:
        print("Opción no válida.")
```

Resultado :

```

Gestion Escuela
1. Añadir profesor
2. Modificar profesor
3. Eliminar profesor
4. Añadir alumno
5. Modificar alumno
6. Eliminar alumno
7. Mostrar todo
0. Salir
Elige una opción: 4
ID alumno: akf1
Nombre: luis
Curso: 2
DNI del profesor asignado (opcional): 341A
Alumno añadido correctamente.

Gestion Escuela
1. Añadir profesor
2. Modificar profesor
3. Eliminar profesor
4. Añadir alumno
5. Modificar alumno
6. Eliminar alumno
7. Mostrar todo
0. Salir
Elige una opción: 7

Escuela: IES Fasuti Barbera (Alaquas) | Resp: Faustí Barbera ---
Profesores:
- DNI: 341A | Nombre: Alex | Tipo: mixto
Alumnos:
- ID: akf1 | Nombre: luis | Curso: 2 | Tutor: Alex

```

Actividad 6

```

from sqlalchemy import create_engine, Column, Integer, String,
ForeignKey, Table
from sqlalchemy.orm import declarative_base, relationship,
sessionmaker

# Base para definir los modelos
Base = declarative_base()

# Tabla de asociación para la relación muchos a muchos
alumno_asignatura = Table(
    'alumno_asignatura',
    Base.metadata,
    Column('alumno_id', Integer, ForeignKey('alumnos.id'),
primary_key=True),
    Column('asignatura_id', Integer, ForeignKey('asignaturas.id'),
primary_key=True)
)

# Modelo para la tabla de asignaturas

```

```
class Asignatura(Base):
    __tablename__ = 'asignaturas'

    id = Column(Integer, primary_key=True, autoincrement=True)
    nombre = Column(String(100), nullable=False)
    alumnos = relationship("Alumno", secondary=alumno_asignatura,
back_populates="asignaturas")

# Modelo para la tabla de alumnos
class Alumno(Base):
    __tablename__ = 'alumnos'

    id = Column(Integer, primary_key=True, autoincrement=True)
    nombre = Column(String(50), nullable=False)
    apellidos = Column(String(100), nullable=False)
    email = Column(String(100), unique=True, nullable=False)
    asignaturas = relationship("Asignatura",
secondary=alumno_asignatura, back_populates="alumnos")

# Conexión a SQLite y creación de las tablas
engine = create_engine('sqlite:///escuela.db', echo=False)
Base.metadata.create_all(engine)

# Gestor de sesiones para interactuar con la BD
Session = sessionmaker(bind=engine)
session = Session()

if __name__ == "__main__":
    # Consulta para comprobar si ya hay alumnos guardados
    alumnos = session.query(Alumno).all()

    if not alumnos:
        # La primera vez crea y guarda los datos
        asig1 = Asignatura(nombre="Acceso a Datos")
        asig2 = Asignatura(nombre="Desarrollo de Interfaces")

        nuevo_alumno = Alumno(
            nombre="Marc",
            apellidos="Giménez Pérez",
            email="marc.gimenez@email.com",
            asignaturas=[asig1, asig2]
        )

        session.add(nuevo_alumno)
        session.commit()
        print("Datos guardados en la base de datos.")
    else:
        # Recupera los datos guardados en el archivo
```

```

    print("Datos recuperados de la base de datos:")
    for a in alumnos:
        print(f"Alumno: {a.nombre} {a.apellidos} - Email: {a.email}")
        for asig in a.asignaturas:
            print(f"    - Asignatura: {asig.nombre}")

    session.close()

```

importar base de datos

```

from sqlalchemy import create_engine, Column, Integer, String, ForeignKey, Table
from sqlalchemy.orm import declarative_base, relationship, sessionmaker

```

Modelo de clase

```

# Modelo para la tabla de alumnos
class Alumno(Base): 2 usages
    __tablename__ = 'alumnos'

    id = Column(Integer, primary_key=True, autoincrement=True)
    nombre = Column(String(50), nullable=False)
    apellidos = Column(String(100), nullable=False)
    email = Column(String(100), unique=True, nullable=False)
    asignaturas = relationship(argument: "Asignatura", secondary=alumno_asignatura, back_populates=

```

Conexión de sqlite y gestor de sesiones

```

# Conexión a SQLite y creación de las tablas en disco
engine = create_engine('sqlite:///escuela.db', echo=False)
Base.metadata.create_all(engine)

# Gestor de s Sesiones para interactuar con la BD
Session = sessionmaker(bind=engine)
session = Session()

```


Actividad 7

```
def es_palindromo(numero):
    """
    >>> es_palindromo(121)
    True
    >>> es_palindromo(123)
    False
    """
    texto = str(numero).strip()
    return texto == texto[::-1]

def es_primo(numero):
    """
    >>> es_primo(7)
    True
    >>> es_primo(4)
    False
    """
    try:
        n = int(numero)
    except ValueError:
        return False
    if n < 2:
        return False

    for i in range(2, n):
        if n % i == 0:
            return False
    return True

if __name__ == "__main__":
    # Ejecutar los tests
    import doctest
    resultado_tests = doctest.testmod()

    # Si los tests fallan, no continúa
    if resultado_tests.failed > 0:
        print("Error: Los tests unitarios no se han superado.")
    else:
        print("Tests unitarios superados correctamente.")
```

```
# Pedir los nombres de los archivos por pantalla
archivo_entrada = input("Introduce el nombre del archivo de
entrada: ")
archivo_salida = input("Introduce el nombre del archivo de
salida: ")

contador_palindromos = 0
contador_primos = 0
lista_ambos = []

# Leer archivo de entrada
with open(archivo_entrada, "r") as entrada:
    for linea in entrada:
        linea = linea.strip()
        if not linea:
            continue

        numero = int(linea)

        es_pal = es_palindromo(numero)
        es_pri = es_primo(numero)

        if es_pal:
            contador_palindromos += 1

        if es_pri:
            contador_primos += 1

        if es_pal and es_pri:
            lista_ambos.append(str(numero))

# Escribir archivo de salida
with open(archivo_salida, "w") as salida:
    salida.write(f"Hay {contador_palindromos} números
palíndromos.\n")
    salida.write(f"Hay {contador_primos} números primos.\n")
    for n in lista_ambos:
        salida.write(f"{n}\n")

print("Proceso completado correctamente.")
```

Escribir números y decir si son palíndromos o primos, creando 2 ficheros uno de entrada y otro de salida.

```
def es_palindromo(numero):
    """
    >>> es_palindromo(121)
    True
    >>> es_palindromo(123)
    False
    """
    texto = str(numero).strip()
    return texto == texto[::-1]

def es_primo(numero):
    """
    >>> es_primo(7)
    True
    >>> es_primo(4)
    False
    """
    try:
        n = int(numero)
    except ValueError:
        return False

    if n < 2:
        return False

    for i in range(2, n):
        if n % i == 0:
            return False

    return True
```

```
if __name__ == "__main__":
    # Ejecutar los tests unitarios automáticamente al iniciar
    import doctest
    resultado_tests = doctest.testmod()

    # Si los tests fallan, no continúa
    if resultado_tests.failed > 0:
        print("Error: Los tests unitarios no se han superado.")
    else:
        print("Tests unitarios superados correctamente.")

    # Pedir los nombres de los archivos por pantalla
    archivo_entrada = input("Introduce el nombre del archivo de entrada: ")
    archivo_salida = input("Introduce el nombre del archivo de salida: ")

    contador_palindromos = 0
    contador_primos = 0
    lista_ambos = []

    # Leer archivo de entrada
    with open(archivo_entrada, "r") as entrada:
        for linea in entrada:
            linea = linea.strip()
            if not linea:
                continue

            numero = int(linea)

            es_pal = es_palindromo(numero)
            es_pri = es_primo(numero)

            if es_pal:
                contador_palindromos += 1

            if es_pri:
                contador_primos += 1

            if es_pal and es_pri:
                lista_ambos.append(str(numero))

    # Escribir archivo de salida
    with open(archivo_salida, "w") as salida:
        salida.write(f"Hay {contador_palindromos} números palindromos.\n")
        salida.write(f"Hay {contador_primos} números primos.\n")
        for n in lista_ambos:
            salida.write(f"{n}\n")

    print("Proceso completado correctamente.")
```

Resultado :

```
Introduce los números que quieras (escribe 'fin' para terminar):
> 1
> 2
> 3
> 4
> 5
> 13
> 11
> 14
> 15
> fin
```

```
1
2
3
4
5
13
11
14
15
```

```
Hi han 6 números palíndroms.
Hi han 5 números cosins.
2
3
5
11
```

```
≡ exempleEixida.txt
```

```
≡ exempleEntrada.txt
```

Actividad 8

```
import shutil
from pathlib import Path

def organizar_archivos(extensiones: list[str]):
    # Directorio de ejecución
    directorio_actual = Path.cwd()

    for ext in extensiones:
        # Limpiar extensión (quitar punto inicial)
        ext_clean = ext.strip().lstrip(".")

        if not ext_clean:
            continue

        # Carpeta destino
        carpeta_destino = directorio_actual / ext_clean

        # Buscar archivos
        pattern = f"*.{ext_clean}"
        archivos = [
            f
            for f in directorio_actual.glob(pattern)
            if f.is_file() and f.suffix.lower() ==
f".{ext_clean.lower()}"
        ]

        if archivos:
```

```
# Crear carpeta destino
carpeta_destino.mkdir(exist_ok=True)

for archivo in archivos:
    destino_archivo = carpeta_destino / archivo.name
    try:
        shutil.move(str(archivo), str(destino_archivo))
        print(
            f"Movido: {archivo.name} ->
{ext_clean}/{archivo.name}"
        )
    except Exception as e:
        print(
            f"Error al mover {archivo.name}: {e}"
        )

if __name__ == "__main__":
    # Extensiones a organizar
    lista_extensiones = ["png", "mp4", "doc", "pdf", "txt"]

    print("Iniciando la organización de archivos...")
    organizar_archivos(lista_extensiones)
    print("¡Proceso finalizado con éxito!")
```

Importar shutil y path

```
import shutil
from pathlib import Path
```

Organizar los archivos:

```
def organizar_archivos(extensiones: list[str]): 1 usage
    # Directorio de ejecución
    directorio_actual = Path.cwd()

    for ext in extensiones:
        # Limpiar extensión (quitar punto inicial)
        ext_clean = ext.strip().lstrip(".")

        if not ext_clean:
            continue

        # Carpeta destino
        carpeta_destino = directorio_actual / ext_clean

        # Buscar archivos
        pattern = f"*.{ext_clean}"
        archivos = [
            f
            for f in directorio_actual.glob(pattern)
            if f.is_file() and f.suffix.lower() == f".{ext_clean.lower()}"
        ]

        if archivos:
            # Crear carpeta destino
            carpeta_destino.mkdir(exist_ok=True)

            for archivo in archivos:
                destino_archivo = carpeta_destino / archivo.name
                try:
                    shutil.move(str(archivo), str(destino_archivo))
                    print(
                        f"Movido: {archivo.name} -> {ext_clean}/{archivo.name}"
                    )
                except Exception as e:
                    print(
                        f"Error al mover {archivo.name}: {e}"
                    )
```

organizar las extensiones

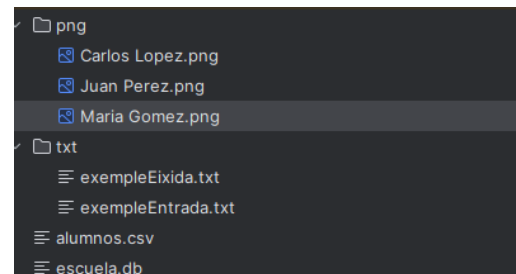
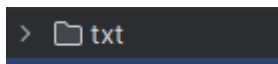
```

if __name__ == "__main__":
    # Extensiones a organizar
    lista_extensiones = ["png", "mp4", "doc", "pdf", "txt"]

    print("Iniciando la organización de archivos...")
    organizar_archivos(lista_extensiones)
    print("¡Proceso finalizado con éxito!")

```

Resultado:



Actividad 9

```

import csv
import sys
from pathlib import Path
import barcode
from barcode.writer import ImageWriter

def formatear_id_ean13(id_texto: str) -> str:
    """Limpia el ID y lo ajusta a 12 dígitos para ser compatible con
    EAN-13."""
    id_limpio = id_texto.strip().strip('"').strip("'")

    if not id_limpio.isdigit():
        raise ValueError(
            f"El id '{id_texto}' solo puede contener números."
        )

    if len(id_limpio) < 12:
        return id_limpio.zfill(12)
    elif len(id_limpio) in (12, 13):
        return id_limpio

```

```
else:
    raise ValueError(
        f"El id '{id_texto}' supera los 13 dígitos permitidos."
    )

def generar_codigos_alumnos(ruta_csv: str):
    path_csv = Path(ruta_csv)

    # Validar existencia del archivo
    if not path_csv.is_file():
        print(
            f"Error: No se encontró el archivo '{ruta_csv}'.",
            file=sys.stderr
        )
        sys.exit(1)

    # Cargar formato EAN13 para imágenes
    EAN13 = barcode.get_barcode_class("ean13")

    with open(path_csv, mode="r", encoding="utf-8") as f:
        lector = csv.reader(f)

        for num_linea, fila in enumerate(lector, start=1):
            # Ignorar líneas vacías
            if not fila:
                continue

            # Validar columnas
            if len(fila) < 2:
                print(
                    f"[Línea {num_linea}] Formato incorrecto. Debe ser 'nombre, id'."
                )
                continue

            nombre_alumno = fila[0].strip().strip('"')
            id_raw = fila[1].strip().strip('"')

            try:
                # Ajustar id a 12 dígitos
                id_valido = formatear_id_ean13(id_raw)

                # Generar código de barras en PNG
                codigo_barras = EAN13(id_valido,
writer=ImageWriter())

                # Guardar imagen PNG
```

```
        nombre_archivo = nombre_alumno
        ruta_generada = codigo_barras.save(nombre_archivo)

        print(
            f"Generado: {ruta_generada} | Alumno:
{nombre_alumno} | Código: {codigo_barras.get_fullcode()}"
        )

    except Exception as e:
        print(
            f"Error en la línea {num_linea}
({nombre_alumno}): {e}",
            file=sys.stderr,
        )

if __name__ == "__main__":
    # Comprobar argumento CSV en consola
    if len(sys.argv) < 2:
        print("Uso del programa:")
        print("  python generar_codigos.py <archivo.csv>")
        sys.exit(1)

    archivo_argumento = sys.argv[1]
    generar_codigos_alumnos(archivo_argumento)
```

importar librerías:

```
import csv
import sys
from pathlib import Path
import barcode
from barcode.writer import ImageWriter
```

método para formatear id ean13:

```
def formatear_id_ean13(id_texto: str) -> str: 1 usage
    """Limpia el ID y lo ajusta a 12 dígitos para ser compatible con EAN-13."""
    id_limpio = id_texto.strip().strip(' ').strip(' ')

    if not id_limpio.isdigit():
        raise ValueError(
            f"El id '{id_texto}' solo puede contener números."
        )

    if len(id_limpio) < 12:
        return id_limpio.zfill(12)
    elif len(id_limpio) in (12, 13):...
    else:
        raise ValueError(
            f"El id '{id_texto}' supera los 13 dígitos permitidos."
        )
```

método para generar códigos de alumnos:

```
def generar_codigos_alumnos(ruta_csv: str): 1 usage
    path_csv = Path(ruta_csv)

    # Validar existencia del archivo
    if not path_csv.is_file():
        print(
            f"Error: No se encontró el archivo '{ruta_csv}'.", file=sys.stderr
        )
        sys.exit(1)

    # Cargar formato EAN13 para imágenes
    EAN13 = barcode.get_barcode_class("ean13")

    with open(path_csv, mode="r", encoding="utf-8") as f:
        lector = csv.reader(f)

        for num_linea, fila in enumerate(lector, start=1):
            # Ignorar líneas vacías
            if not fila:
                continue

            # Validar columnas
            if len(fila) < 2:
                print(
                    f"[Línea {num_linea}] Formato incorrecto. Debe ser 'nombre, id'."
                )
                continue
```

```

nombre_alumno = fila[0].strip().strip('')
id_raw = fila[1].strip().strip('')

try:
    # Ajustar ID a 12 dígitos
    id_valido = formatear_id_ean13(id_raw)

    # Generar código de barras en PNG
    codigo_barras = EAN13(id_valido, writer=ImageWriter())

    # Guardar imagen PNG
    nombre_archivo = nombre_alumno
    ruta_generada = codigo_barras.save(nombre_archivo)

    print(
        f"Generado: {ruta_generada} | Alumno: {nombre_alumno} | Código: {codigo_barras.get_fullcode()}"
    )

except Exception as e:
    print(
        f"Error en la línea {num_linea} ({nombre_alumno}): {e}",
        file=sys.stderr,
    )

```

Comprobar argumento csv :

```

if __name__ == "__main__":
    # Comprobar argumento CSV en la consola
    if len(sys.argv) < 2:
        print("Uso del programa:")
        print(" python generar_codigos.py <archivo.csv>")
        sys.exit(1)

    archivo_argumento = sys.argv[1]
    generar_codigos_alumnos(archivo_argumento)

```

Resultado:

≡ alumnos.csv

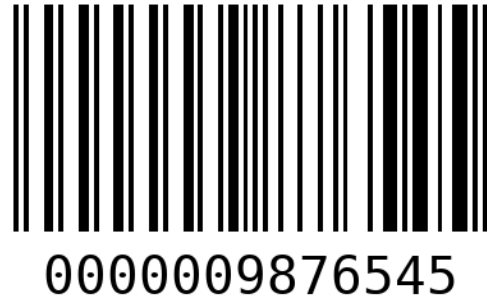
```

Juan Perez, 12345
Maria Gomez, 987654
Carlos Lopez, 456

```

```
(.venv) PS C:\Users\gokex\PycharmProjects\PythonProject1> python UD00-PE9.py alumnos.csv
Generado: Juan Perez.png | Alumno: Juan Perez | Código: 0000000123457
Generado: María Gomez.png | Alumno: Maria Gomez | Código: 0000009876545
Generado: Carlos Lopez.png | Alumno: Carlos Lopez | Código: 0000000004565
(.venv) PS C:\Users\gokex\PycharmProjects\PythonProject1> █
```

 Carlos Lopez.png
 Juan Perez.png
 Maria Gomez.png



Actividad 10

```
import sys
from pathlib import Path
from PIL import Image
from pyzbar.pyzbar import decode

def leer_codigos_directorio(directorio: str = "."):
    ruta_dir = Path(directorio)

    # Validar existencia del directorio
    if not ruta_dir.is_dir():
        print(
            f"Error: El directorio '{directorio}' no existe.",
            file=sys.stderr
        )
        sys.exit(1)

    # Obtener archivos .png
    archivos_png = sorted(list(ruta_dir.glob("*.png")))

    if not archivos_png:
        print(f"No se encontraron archivos .png en '{ruta_dir}'.")
```

```
        return

    print(f"Se encontraron {len(archivos_png)} archivos png.\n")
    print(f"{'Nombre Alumno':<25} | {'ID del alumno'}")
    print("-" * 50)

    for archivo in archivos_png:
        # Nombre del alumno desde el archivo
        nombre_alumno = archivo.stem

        try:
            # Abrir imagen
            imagen = Image.open(archivo)

            # Leer códigos de barras
            codigos_detectados = decode(imagen)

            if not codigos_detectados:
                print(
                    f"{'nombre_alumno:<25} | [No se detectó ningún  
código de barras]"
                )
                continue

            for codigo in codigos_detectados:
                # Extraer texto del código
                dato_raw = codigo.data.decode("utf-8")

                # Limpiar ceros a la izquierda y quitar el dígito
                # final (es de control)
                id_limpio = dato_raw.lstrip("0")
                if len(id_limpio) > 1:
                    id_limpio = id_limpio[:-1]

                print(f"{'nombre_alumno:<25} | {id_limpio}")

            except Exception as e:
                print(f"{'nombre_alumno:<25} | Error al leer el archivo:  
{e}")

if __name__ == "__main__":
    # Usar ruta enviada por consola o el directorio actual
    dir_input = sys.argv[1] if len(sys.argv) > 1 else "."
    leer_codigos_directorio(dir_input)
```

importamos las bibliotecas:

```
import sys
from pathlib import Path
from PIL import Image
from pyzbar.pyzbar import decode
```

método para leer el código:

```
def leer_codigos_directorio(directorio: str = "."): 1 usage
    ruta_dir = Path(directorio)

    # Validar existencia del directorio
    if not ruta_dir.is_dir():
        print(
            f"Error: El directorio '{directorio}' no existe.", file=sys.stderr
        )
        sys.exit(1)

    # Obtener archivos .png
    archivos_png = sorted(list(ruta_dir.glob("*.png")))

    if not archivos_png:
        print(f"No se encontraron archivos .png en '{ruta_dir}'")
        return

    print(f"Se encontraron {len(archivos_png)} archivos PNG. Leyendo códigos...\n")
    print(f"{'Nombre Alumno':<25} | {'ID del alumno'}")
    print("-" * 50)

    for archivo in archivos_png:
        # Nombre del alumno desde el archivo (sin extensión)
        nombre_alumno = archivo.stem

        try:
            # Abrir imagen
            imagen = Image.open(archivo)

            # Leer códigos de barras
            codigos_detectados = decode(imagen)

            if not codigos_detectados:
                print(
                    f"{'nombre_alumno:<25'} | [No se detectó ningún código de barras]"
                )
                continue
```

```

for codigo in codigos_detectados:
    # Extraer texto del código
    dato_raw = codigo.data.decode("utf-8")

    # Limpiar ceros a la izquierda y quitar el dígito de control final
    id_limpio = dato_raw.lstrip("0")
    if len(id_limpio) > 1:
        id_limpio = id_limpio[:-1]

    print(f"{nombre_alumno:<25} | {id_limpio}")

except Exception as e:
    print(f"{nombre_alumno:<25} | Error al leer el archivo: {e}")

```

metodo para usar la ruta:

```

if __name__ == "__main__":
    # Usar ruta enviada por consola o el directorio actual
    dir_input = sys.argv[1] if len(sys.argv) > 1 else "."
    leer_codigos_directorio(dir_input)

```

Resultado:

Nombre Alumno	ID del alumno
Carlos Lopez	456
Juan Perez	12345
Maria Gomez	987654

Actividad 11

```
import csv
import hashlib
from kivy.app import App
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.button import Button
from kivy.uix.label import Label
from kivy.uix.textinput import TextInput

# Clase para organizar la pantalla de login
class VentanaLogin(BoxLayout):

    def __init__(self, **kwargs):
        super().__init__(**kwargs)

        # Ponemos los elementos uno debajo de otro
        self.orientation = "vertical"
        self.padding = 20
        self.spacing = 10

        # Guardaremos aquí los usuarios y sus contraseñas
        self.usuarios = {}

        # Leemos el archivo CSV al arrancar el programa
        self.cargar_usuarios_csv()

        # Texto y caja para escribir el usuario
        self.add_widget(Label(text="Usuario:"))
        self.input_usuario = TextInput(multiline=False)
        self.add_widget(self.input_usuario)

        # Texto y caja para la contraseña (oculta al escribir)
        self.add_widget(Label(text="Contraseña:"))
        self.input_password = TextInput(multiline=False, password=True)
        self.add_widget(self.input_password)

        # Botón que llama a la función de comprobar cuando lo pulsas
        self.boton_comprobar = Button(text="Comprobar")
        self.boton_comprobar.bind(on_press=self.comprobar_login)
```

```
self.add_widget(self.boton_comprobar)

# Texto final donde saldrá OK o ERROR
self.label_resultado = Label(text="", font_size="20sp")
self.add_widget(self.label_resultado)

def cargar_usuarios_csv(self):
    """Lee el fichero users.csv y guarda los datos en memoria"""
    try:
        with open("users.csv", mode="r", encoding="utf-8") as
archivo:
            lineas = csv.reader(archivo, delimiter=":")
            for fila in lineas:
                if len(fila) == 2:
                    usuario = fila[0].strip().replace("'", "")
                    hash_shal = fila[1].strip().replace("'", "")
                    self.usuarios[usuario] = hash_shal
    except FileNotFoundError:
        print("No se ha encontrado el archivo users.csv")

def calcular_shal(self, texto):
    """Encripta un texto en formato SHA1"""
    hash_objeto = hashlib.shal(texto.encode("utf-8"))
    return hash_objeto.hexdigest()

def comprobar_login(self, instance):
    """Revisa si el usuario existe y si la clave encriptada
coincide"""
    usuario_ingresado = self.input_usuario.text.strip()
    password_ingresada = self.input_password.text.strip()

    # Encriptamos la clave introducida
    password_shal = self.calcular_shal(password_ingresada)

    # Comprobamos si el usuario existe y la clave es correcta
    if (
        usuario_ingresado in self.usuarios
        and self.usuarios[usuario_ingresado] == password_shal
    ):
        self.label_resultado.text = "OK"
        self.label_resultado.color = (0, 1, 0, 1) # Texto en verde
    else:
        self.label_resultado.text = "ERROR"
```

```
        self.label_resultado.color = (1, 0, 0, 1) # Texto en rojo

# Clase principal de Kivy para lanzar la app
class AplicacionLogin(App):

    def build(self):
        self.title = "Control de Acceso"
        return VentanaLogin()

if __name__ == "__main__":
    AplicacionLogin().run()
```

importar clases: (si no funciona es porque se necesita instalar una versión anterior en python)

```
import csv
import hashlib
from kivy.app import App
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.button import Button
from kivy.uix.label import Label
from kivy.uix.textinput import TextInput
```

clase login:

```
# Clase para organizar la pantalla de login
class VentanaLogin(BoxLayout):

    def __init__(self, **kwargs):
        super().__init__(**kwargs)

        # Ponemos los elementos uno debajo de otro
        self.orientation = "vertical"
        self.padding = 20
        self.spacing = 10

        # Guardaremos aquí los usuarios y sus contraseñas
        self.usuarios = {}

        # Leemos el archivo CSV al arrancar el programa
        self.cargar_usuarios_csv()

        # Texto y caja para escribir el usuario
        self.add_widget(Label(text="Usuario:"))
        self.input_usuario = TextInput(multiline=False)
        self.add_widget(self.input_usuario)

        # Texto y caja para la contraseña (oculta al escribir)
        self.add_widget(Label(text="Contraseña:"))
        self.input_password = TextInput(multiline=False, password=True)
        self.add_widget(self.input_password)

        # Botón que llama a la función de comprobar cuando lo pulsas
        self.boton_comprobar = Button(text="Comprobar")
        self.boton_comprobar.bind(on_press=self.comprobar_login)
        self.add_widget(self.boton_comprobar)

        # Texto final donde saldrá OK o ERROR
        self.label_resultado = Label(text="", font_size="20sp")
        self.add_widget(self.label_resultado)
```

```

def cargar_usuarios_csv(self):
    """Lee el fichero users.csv y guarda los datos en memoria"""
    try:
        with open("users.csv", mode="r", encoding="utf-8") as archivo:
            lineas = csv.reader(archivo, delimiter=",")
            for fila in lineas:
                if len(fila) == 2:
                    usuario = fila[0].strip().replace("'", "")
                    hash_sha1 = fila[1].strip().replace("'", "")
                    self.usuarios[usuario] = hash_sha1
    except FileNotFoundError:
        print("No se ha encontrado el archivo users.csv")

def calcular_sha1(self, texto):
    """Encripta un texto en formato SHA1"""
    hash_objeto = hashlib.sha1(texto.encode("utf-8"))
    return hash_objeto.hexdigest()

def comprobar_login(self, instance):
    """Revisa si el usuario existe y si la clave encriptada coincide"""
    usuario_ingresado = self.input_usuario.text.strip()
    password_ingresada = self.input_password.text.strip()

    # Encriptamos la clave introducida
    password_sha1 = self.calcular_sha1(password_ingresada)

    # Comprobamos si el usuario existe y la clave es correcta
    if (
        usuario_ingresado in self.usuarios
        and self.usuarios[usuario_ingresado] == password_sha1
    ):
        self.label_resultado.text = "OK"
        self.label_resultado.color = (0, 1, 0, 1) # Texto en verde
    else:
        self.label_resultado.text = "ERROR"
        self.label_resultado.color = (1, 0, 0, 1) # Texto en rojo

```

Clase para kivy y lanzar app:

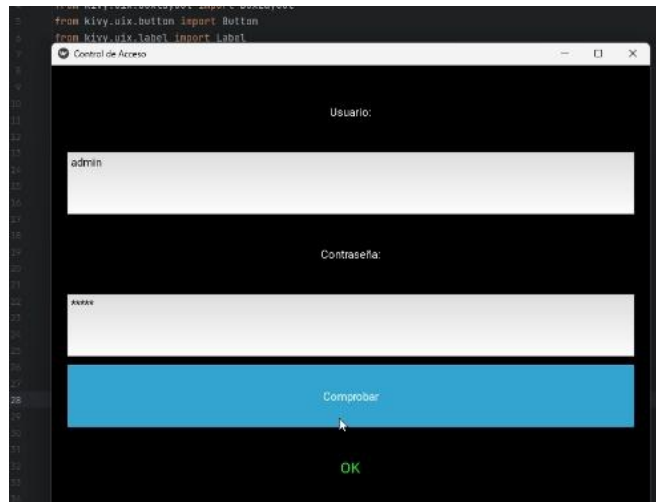
```

# Clase principal de Kivy para lanzar la app
class AplicacionLogin(App):

    def build(self):
        self.title = "Control de Acceso"
        return VentanaLogin()

if __name__ == "__main__":
    AplicacionLogin().run()

```

Resultado:

Actividad 12

```
import ssl

# Desactivamos la verificación SSL para que EasyOCR pueda descargar los
modelos sin error
ssl._create_default_https_context = ssl._create_unverified_context

import cv2
import easyocr
import matplotlib.pyplot as plt

# Lector indicando los idiomas que queremos buscar
lector = easyocr.Reader(['es', 'en'])

# Cargamos la imagen que queremos leer
ruta_imagen = 'imagen_ejemplo.png'
imagen = cv2.imread(ruta_imagen)

if imagen is None:
    raise FileNotFoundError(f"No se pudo encontrar la imagen en la ruta:
{ruta_imagen}")

# Leemos todo el texto que aparece en la imagen
resultados = lector.readtext(imagen)

# Mostramos en pantalla el texto encontrado y su porcentaje de
seguridad
```

```

print(" Textos detectados en la imagen")
for (ubicacion, texto, seguridad) in resultados:
    print(f"- Texto leído: '{texto}' (Seguridad: {seguridad *
100:.0f}%)")

    # Cuadro verde alrededor del texto que hemos encontrado
    punto1 = (int(ubicacion[0][0]), int(ubicacion[0][1]))
    punto2 = (int(ubicacion[2][0]), int(ubicacion[2][1]))
    cv2.rectangle(imagen, punto1, punto2, (0, 255, 0), 2)

# Mostramos la imagen final con los recuadros verdes encima del texto
imagen_color_correcte = cv2.cvtColor(imagen, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(8, 8))
plt.imshow(imagen_color_correcte)
plt.axis('off')
plt.title("Resultado del reconocimiento de texto")
plt.show()

```

importar bibliotecas :

```

import ssl
import cv2
import easyocr
import matplotlib.pyplot as plt
# Desactivar la verificación ssl para que no de error
ssl._create_default_https_context = ssl._create_unverified_context

```

lector, cargar imagen y leer el texto de la imagen:

```

# Lector indicando los idiomas que queremos buscar
lector = easyocr.Reader(['es', 'en'])

# Cargamos la imagen que queremos leer
ruta_imagen = 'imagen_ejemplo.png'
imagen = cv2.imread(ruta_imagen)

if imagen is None:
    raise FileNotFoundError(f"No se pudo encontrar la imagen en la ruta: {ruta_imagen}")

# Leemos todo el texto que aparece en la imagen
resultados = lector.readtext(imagen)

```

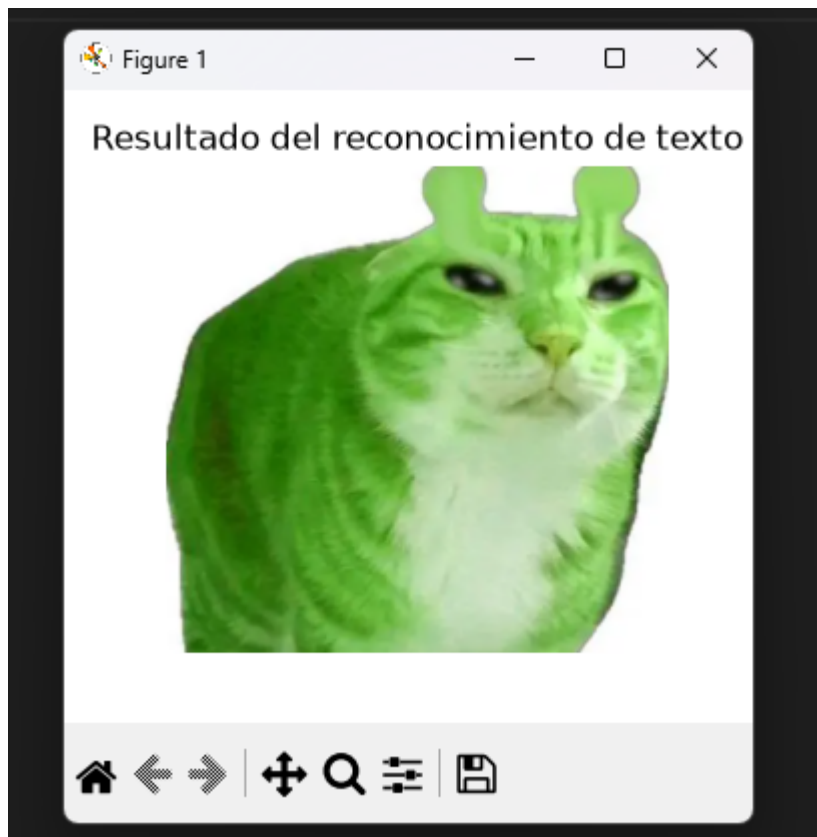
mostrar texto:

```
# Mostramos en pantalla el texto encontrado y su porcentaje de seguridad
print(" Textos detectados en la imagen")
for (ubicacion, texto, seguridad) in resultados:
    print(f"- Texto leído: '{texto}' (Seguridad: {seguridad * 100:.0f}%)")

# Cuadro verde alrededor del texto que hemos encontrado
punto1 = (int(ubicacion[0][0]), int(ubicacion[0][1]))
punto2 = (int(ubicacion[2][0]), int(ubicacion[2][1]))
cv2.rectangle(imagen, punto1, punto2, (0, 255, 0), 2)

# Mostramos la imagen final con los recuadros verdes encima del texto
imagen_color_correcte = cv2.cvtColor(imagen, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(8, 8))
plt.imshow(imagen_color_correcte)
plt.axis('off')
plt.title("Resultado del reconocimiento de texto")
plt.show()
```

Resultado (sin texto, no tenía otra imagen a mano):



Actividad 13

```
import requests

# Pedimos la especie al usuario por teclado
especie = input("Introduce la especie de los personajes (Human, Alien...): ")

# URL de la API de Rick and Morty filtrando por especie
url = f"https://rickandmortyapi.com/api/character/?species={especie}"

# Hacemos la petición a la API
respuesta = requests.get(url)

# Comprobamos si la respuesta es correcta (código 200)
if respuesta.status_code == 200:
    # Convertimos la respuesta que viene de la api a formato json como a
    un diccionario
    datos = respuesta.json()

    # Extraemos la lista de personajes
    personajes = datos.get("results", [])

    print(f"\nPersonajes encontrados de la especie '{especie}' ")
    # Recorremos cada personaje y mostramos su nombre y estado
    for personaje in personajes:
        print(f"- {personaje['name']} (Estado: {personaje['status']})")

elif respuesta.status_code == 404:
    print(f"\nNo se han encontrado personajes para la especie '{especie}'.")
else:
    print("\nHa ocurrido un error al conectar con la API.")
```

importamos las bibliotecas:

```
import requests
```

Pedimos la especie por teclado, accedemos a la api y mandamos petición a la api:

```
# Pedimos la especie al usuario por teclado
especie = input("Introduce la especie de los personajes (Human, Alien...): ")

# URL de la API de Rick and Morty filtrando por especie
url = f"https://rickandmortyapi.com/api/character/?species={especie}"

# Hacemos la petición a la API
respuesta = requests.get(url)
```

comprobar si la respuesta es correcta e imprimir la información:

```
# Comprobamos si la respuesta es correcta (código 200)
if respuesta.status_code == 200:
    # Convertimos la respuesta que viene de la api a formato json como a un diccionario
    datos = respuesta.json()

    # Extraemos la lista de personajes
    personajes = datos.get("results", [])

    print(f"\nPersonajes encontrados de la especie '{especie}' ")
    # Recorremos cada personaje y mostramos su nombre y estado
    for personaje in personajes:
        print(f"- {personaje['name']} (Estado: {personaje['status']})")

elif respuesta.status_code == 404:
    print(f"\nNo se han encontrado personajes para la especie '{especie}'.")
else:
    print("\nHa ocurrido un error al conectar con la API.")
```

Resultado:

```
PS C:\Users\1alex\OneDrive\Documentos\PYTHONUD00-P2> & C:/Users/1alex/A
Introduce la especie de los personajes (Human, Alien...): Monstruo

No se han encontrado personajes para la especie 'Monstruo'.
PS C:\Users\1alex\OneDrive\Documentos\PYTHONUD00-P2> & C:/Users/1alex/A
Introduce la especie de los personajes (Human, Alien...): Humano

    Personajes encontrados de la especie 'Humano'
- Aqua Morty (Estado: unknown)
- Aqua Rick (Estado: unknown)
- Chris (Estado: Alive)
- Concerto (Estado: Dead)
- Cyclops Morty (Estado: Alive)
- Cyclops Rick (Estado: Dead)
- Dr. Xenon Bloom (Estado: Dead)
- Evil Rick (Estado: Dead)
- Hammerhead Morty (Estado: unknown)
- Kyle (Estado: Dead)
- Lady Katana (Estado: Dead)
- Little Dipper (Estado: Alive)
- Lizard Morty (Estado: Alive)
- Michael Thompson (Estado: Alive)
- Morty Jr. (Estado: Alive)
- Mr. Meeseeks (Estado: unknown)
- Mr. Needful (Estado: Alive)
```

Actividad 14

```
def es_valida(tablero, fila, columna):
    """Comprueba si es seguro colocar una reina en la fila y columna
    x"""
    for i in range(fila):
        # Revisa si hay otra reina en la misma columna
        if tablero[i] == columna:
            return False
        # Revisa si hay otra reina en las diagonales
        if abs(tablero[i] - columna) == abs(i - fila):
            return False
    return True

def resolver_n_reinas(fila, tablero, n):
    ##Calcula recursivamente las soluciones posibles
    if fila == n:
        # Se han colocado todas las reinas con éxito
        return 1

    soluciones = 0
    for columna in range(n):
        if es_valida(tablero, fila, columna):
            tablero[fila] = columna
            # Continúa probando en la siguiente fila
            soluciones += resolver_n_reinas(fila + 1, tablero, n)

    return soluciones

def obtener_soluciones_totales(n):
    # Inicia el cálculo para un tamaño N
    tablero = [-1] * n
    return resolver_n_reinas(0, tablero, n)

# Prueba
if __name__ == "__main__":
    print(f"{'Tamaño N':<10} | {'Soluciones totales':<20}")
    print("-" * 35)

    for n in range(4, 11):
```

```
total = obtener_soluciones_totales(n)
print(f"{n:<10} | {total:<20}")
```

Comprobación de colocación:

```
def es_valida(tablero, fila, columna):
    """Comprueba si es seguro colocar una reina en la fila y columna x"""
    for i in range(fila):
        # Revisa si hay otra reina en la misma columna
        if tablero[i] == columna:
            return False
        # Revisa si hay otra reina en las diagonales
        if abs(tablero[i] - columna) == abs(i - fila):
            return False
    return True
```

Devolver reinas:

```
def resolver_n_reinas(fila, tablero, n):
    ##Calcula recursivamente las soluciones posibles
    if fila == n:
        # Se han colocado todas las reinas con éxito
        return 1

    soluciones = 0
    for columna in range(n):
        if es_valida(tablero, fila, columna):
            tablero[fila] = columna
            # Continúa probando en la siguiente fila
            soluciones += resolver_n_reinas(fila + 1, tablero, n)

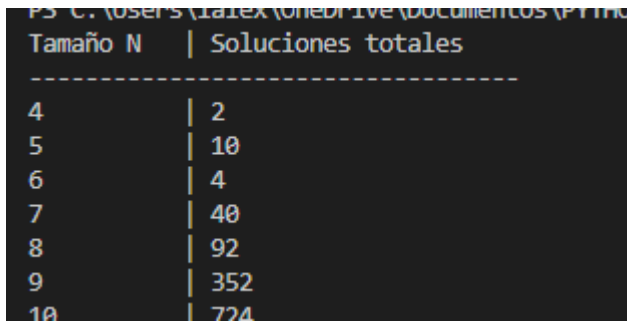
    return soluciones
```

Inicia el cálculo y ejecutar el código:

```
def obtener_soluciones_totales(n):
    # Inicia el cálculo para un tamaño N
    tablero = [-1] * n
    return resolver_n_reinas(0, tablero, n)

# Prueba
if __name__ == "__main__":
    print(f"{ 'Tamaño N':<10} | { 'Soluciones totales':<20}")
    print("-" * 35)

    for n in range(4, 11):
        total = obtener_soluciones_totales(n)
        print(f"{n:<10} | {total:<20}")
```

Resultado:


Tamaño N	Soluciones totales
4	2
5	10
6	4
7	40
8	92
9	352
10	724

Actividad 15

```

import sys
import time
import re
import pyperclip

# Lee el archivo de texto y devuelve una lista con las palabras
prohibidas
def cargar_palabras_prohibidas(fichero):
    try:
        with open(fichero, 'r', encoding='utf-8') as f:
            # Leemos cada línea, quita espacios sobrantes y elimina
            # líneas vacías
            return [linea.strip() for linea in f if linea.strip()]
    except FileNotFoundError:
        print(f"Error: No se pudo encontrar el archivo '{fichero}'.")
        sys.exit(1)

# Sustituye las palabras prohibidas del texto por asteriscos
def censurar_texto(texto, palabras_prohibidas):
    texto_censurado = texto
    for palabra in palabras_prohibidas:
        # Buscamos la palabra exacta sin diferenciar mayúsculas y
        # minúsculas
        patron = re.compile(rf'\b{re.escape(palabra)}\b', re.IGNORECASE)
        # Reemplaza la palabra por asteriscos según su longitud
        texto_censurado = patron.sub('*' * len(palabra),
        texto_censurado)
    return texto_censurado

def main():
    # Comprueba que se pase el archivo por parámetro en la consola

```

```
if len(sys.argv) < 2:
    print("Uso: python programa.py <archivo_de_palabras.txt>")
    sys.exit(1)

fichero_palabras = sys.argv[1]
palabras_prohibidas = cargar_palabras_prohibidas(fichero_palabras)

print(f"Programa iniciado. Vigilando el portapapeles...")
print("Presiona Ctrl+C para salir.")

# Guarda el último texto copiado para no repetirlo
ultimo_texto = ""

try:
    while True:
        # Obtiene el texto actual del portapapeles
        texto_actual = pyperclip.paste()

        # Si el texto ha cambiado y no está vacío
        if texto_actual != ultimo_texto and texto_actual:
            texto_censurado = censurar_texto(texto_actual,
palabras_prohibidas)

            # Si se detecta palabras prohibidas, actualiza el
portapapeles
            if texto_censurado != texto_actual:
                pyperclip.copy(texto_censurado)
                print("\nTexto censurado detectado:")
                print(f"Original:  {texto_actual}")
                print(f"Censurado: {texto_censurado}")
                ultimo_texto = texto_censurado
            else:
                ultimo_texto = texto_actual

            # Espera medio segundo antes de volver a revisar
            time.sleep(0.5)

except KeyboardInterrupt:
    print("\nPrograma detenido.")

if __name__ == "__main__":
    main()
```

Importar bibliotecas:

```
import sys
import time
import re
import pyperclip
```

Lee el archivo, devuelve una lista con las palabras prohibidas y sustituye las palabras prohibidas por “*” :

```
# Lee el archivo de texto y devuelve una lista con las palabras prohibidas
def cargar_palabras_prohibidas(fichero):
    try:
        with open(fichero, 'r', encoding='utf-8') as f:
            # Leemos cada línea, quita espacios sobrantes y elimina líneas vacías
            return [linea.strip() for linea in f if linea.strip()]
    except FileNotFoundError:
        print(f"Error: No se pudo encontrar el archivo '{fichero}'.")
        sys.exit(1)

# Sustituye las palabras prohibidas del texto por asteriscos
def censurar_texto(texto, palabras_prohibidas):
    texto_censurado = texto
    for palabra in palabras_prohibidas:
        # Buscamos la palabra exacta sin diferenciar mayúsculas y minúsculas
        patron = re.compile(rf'\b{re.escape(palabra)}\b', re.IGNORECASE)
        # Reemplazamos la palabra por asteriscos según su longitud
        texto_censurado = patron.sub('*' * len(palabra), texto_censurado)
    return texto_censurado
```

Comprueba si el archivo pasa por consola y guarda el último texto copiado para no repetirlo :

```
def main():
    # Comprueba que se pase el archivo por parámetro en la consola
    if len(sys.argv) < 2:
        print("Uso: python programa.py <archivo_de_palabras.txt>")
        sys.exit(1)

    fichero_palabras = sys.argv[1]
    palabras_prohibidas = cargar_palabras_prohibidas(fichero_palabras)

    print(f"Programa iniciado. Vigilando el portapapeles...")
    print("Presiona Ctrl+C para salir.")

    # Guarda el último texto copiado para no repetirlo constantemente
    ultimo_texto = ""
```

```
try:
    while True:
        # Obtiene el texto actual del portapapeles
        texto_actual = pyperclip.paste()

        # Si el texto ha cambiado y no está vacío
        if texto_actual != ultimo_texto and texto_actual:
            texto_censurado = censurar_texto(texto_actual, palabras_prohibidas)

            # Si se detecta palabras prohibidas, actualiza el portapapeles
            if texto_censurado != texto_actual:
                pyperclip.copy(texto_censurado)
                print("\nTexto censurado detectado:")
                print(f"Original: {texto_actual}")
                print(f"Censurado: {texto_censurado}")
                ultimo_texto = texto_censurado
            else:
                ultimo_texto = texto_actual

        # Espera medio segundo antes de volver a revisar
        time.sleep(0.5)

except KeyboardInterrupt:
    print("\nPrograma detenido.")

if __name__ == "__main__":
    main()
```

Resultado:

```
Texto censurado detectado:
Original:  HOLA Sergi viva pYThons
Censurado: **** Sergi viva *****
```

```
≡ lista.txt
1  Hola
2  pYThons
```